Omar Ahmed

VPN Tunneling

Tasks:[https://seedsecuritylabs.org/Labs_20.04/Files/VPN_Tunnel/VPN_Tunnel.pdf]

Task 1:

Ip Addresses:

- Host U: 10.9.0.5
- Host V: 192.168.60.5
- VPN Server: 10.9.0.11
    - 192.168.60.11

Host U can communicate with VPN Server:

```
[11/22/24]seed@VM:~/.../Labsetup$ dockps
a0b0e23dfb77  server-router
dcb0604b5a7e  host-192.168.60.5
c5537bdbfb95  host-192.168.60.6
ec5ca555c665  client-10.9.0.5
[11/22/24]seed@VM:~/.../Labsetup$ docksh ec
root@ec5ca555c665:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.067 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.040 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.042 ms
^C
--- 10.9.0.11 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2038ms
rtt min/avg/max/mdev = 0.040/0.049/0.067/0.012 ms
root@ec5ca555c665:/#
```

- Host U can ping to VPN server via IP address 10.9.0.11

VPNServer can communicate with Host V.

```
[11/22/24]seed@VM:~/.../Labsetup$ dockps
a0b0e23dfb77  server-router
dcb0604b5a7e  host-192.168.60.5
c5537bdbfb95  host-192.168.60.6
ec5ca555c665  client-10.9.0.5
[11/22/24]seed@VM:~/.../Labsetup$ docksh a0
root@a0b0e23dfb77:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=64 time=0.069 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=64 time=0.040 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=64 time=0.043 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=64 time=0.044 ms
^C
--- 192.168.60.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3054ms
rtt min/avg/max/mdev = 0.040/0.049/0.069/0.011 ms
root@a0b0e23dfb77:/#
```

- VPN server can ping to Host V via IP address 192.168.60.5

Host U should not be able to communicate with Host V.

```
[11/22/24]seed@VM:~/.../Labsetup$ dockps
a0b0e23dfb77  server-router
dcb0604b5a7e  host-192.168.60.5
c5537bdbfb95  host-192.168.60.6
ec5ca555c665  client-10.9.0.5
[11/22/24]seed@VM:~/.../Labsetup$ docksh ec
root@ec5ca555c665:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^C
--- 192.168.60.5 ping statistics ---
8 packets transmitted, 0 received, 100% packet loss, time 7163ms
```

- As shown, ping is not successful (100% packet loss). This is good, Host U should not be able to communicate with Host V, because the VPN server should be between them

Run tcpdump on the router, and sniff the traffic on each of the network. Show that you can capture packets.

```
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root@a0b0e23dfb77:/# tcpdump -i eth0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
21:30:09.283308 IP 10.9.0.5 > 10.9.0.11: ICMP echo request, id 27, seq 1, length 64
21:30:09.283329 IP 10.9.0.11 > 10.9.0.5: ICMP echo reply, id 27, seq 1, length 64
21:30:10.311599 IP 10.9.0.5 > 10.9.0.11: ICMP echo request, id 27, seq 2, length 64
21:30:10.311623 IP 10.9.0.11 > 10.9.0.5: ICMP echo reply, id 27, seq 2, length 64
21:30:11.336534 IP 10.9.0.5 > 10.9.0.11: ICMP echo request, id 27, seq 3, length 64
21:30:11.336560 IP 10.9.0.11 > 10.9.0.5: ICMP echo reply, id 27, seq 3, length 64
21:30:14.408025 ARP, Request who-has 10.9.0.5 tell 10.9.0.11, length 28
21:30:14.408053 ARP, Request who-has 10.9.0.11 tell 10.9.0.5, length 28
21:30:14.408061 ARP, Reply 10.9.0.11 is-at 02:42:0a:09:00:0b, length 28
21:30:14.408062 ARP, Reply 10.9.0.5 is-at 02:42:0a:09:00:05, length 28
```

```
PING 192.168.60.5 (192.168.60.5) 56(84) b
^C
--- 192.168.60.5 ping statistics ---
8 packets transmitted, 0 received, 100% p

root@ec5ca555c665:/# exit
exit
[11/22/24]seed@VM:~/.../Labsetup$ dockps
a0b0e23dfb77  server-router
dcb0604b5a7e  host-192.168.60.5
c5537bdbfb95  host-192.168.60.6
ec5ca555c665  client-10.9.0.5
[11/22/24]seed@VM:~/.../Labsetup$ docksh
root@ec5ca555c665:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes o
64 bytes from 10.9.0.11: icmp_seq=1 ttl=6
64 bytes from 10.9.0.11: icmp_seq=2 ttl=6
64 bytes from 10.9.0.11: icmp_seq=3 ttl=6
^C
--- 10.9.0.11 ping statistics ---
3 packets transmitted, 3 received, 0% pac
rtt min/avg/max/mdev = 0.066/0.069/0.071/
root@ec5ca555c665:/# exit
exit
```

Task 2:

Task 2.a

```
root@ec5ca555c665:/volumes# chmod a+x tun.py
root@ec5ca555c665:/volumes# tun.py
Interface Name: tun0
```

```
root@ec5ca555c665:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: tun0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOW
    link/none
26: eth0@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqu
    link/ether 02:42:0a:09:00:05 brd ff:ff:ff:ff:ff:ff link-netnsid
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
       valid_lft forever preferred_lft forever
root@ec5ca555c665:/#
```

- Running the original tun.py shown above

```
# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'nguye%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)
```

- Changing iface prefix name

```
History restored

[11/22/24]seed@VM:~/.../Labsetup$ dockps ec
"docker ps" accepts no arguments.
See 'docker ps --help'.

Usage:  docker ps [OPTIONS]

List containers
[11/22/24]seed@VM:~/.../Labsetup$ docksh ec
root@ec5ca555c665:/# tun.py
bash: tun.py: command not found
root@ec5ca555c665:/# cd volumes/
root@ec5ca555c665:/volumes# tun.py
Interface Name: nguye0
```

```
[11/22/24]seed@VM:~/.../Labsetup$ docksh ec
root@ec5ca555c665:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: tun0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DO
    link/none
3: nguye0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state
    link/none
26: eth0@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noq
    link/ether 02:42:0a:09:00:05 brd ff:ff:ff:ff:ff:ff link-netnsid
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
       valid_lft forever preferred_lft forever
root@ec5ca555c665:/#
```

- New tun.py with custom iface prefix

Task 2.b

```
root@ec5ca555c665:/# ip addr add 192.168.53.99/24 dev nguye0
RTNETLINK answers: File exists
root@ec5ca555c665:/# ip link set dev nguye0 up
root@ec5ca555c665:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group defa
    link/none
3: nguye0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group de
    link/none
    inet 192.168.53.99/24 scope global nguye0
       valid_lft forever preferred_lft forever
26: eth0@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:0a:09:00:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
       valid_lft forever preferred_lft forever
root@ec5ca555c665:/#
```

- After doing the configuration commands, iface nguye0 now have an ip address 192.168.53.99/24

Task 2.c

On Host U, ping a host in the 192.168.53.0/24 network. What are printed out by the tun.py program? What has happened? Why?

```
root@ec5ca555c665:/# ping 192.168.53.1
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.
^C
--- 192.168.53.1 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4080ms
```

```
IP / ICMP 192.168.53.99 > 192.168.53
IP / ICMP 192.168.53.99 > 192.168.53
IP / ICMP 192.168.53.99 > 192.168.53
IP / ICMP 192.168.53.99 > 192.168.53
IP / ICMP 192.168.53.99 > 192.168.53
IP / ICMP 192.168.53.99 > 192.168.53
```

- The ping fails and there is 100% packet loss. This is because the other side of the tunnel has not been configured yet, so any pings to external IP addresses will not work.

On Host U, ping a host in the internal network 192.168.60.0/24, Does tun.py print out

anything? Why?

```
root@ec5ca555c665:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^C
--- 192.168.60.5 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1023ms
```

- tun.py does not print out anything
- This is because ???

Task 2.d

```
root@ec5ca555c665:/# ping 192.168.53.100 -c1
PING 192.168.53.100 (192.168.53.100) 56(84) bytes of data.
64 bytes from 192.168.53.100: icmp_seq=1 ttl=64 time=1.39 ms

--- 192.168.53.100 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.393/1.393/1.393/0.000 ms
```

```
  File "./tun.py", line 44, in <module>
    os.write(tun, b'This is some arbitrary data')
OSError: [Errno 22] Invalid argument
root@ec5ca555c665:/volumes# tun.py
Interface Name: nguye0
IP / ICMP 192.168.53.99 > 192.168.53.100 echo-request 0 /
Sent ICMP echo reply: IP / ICMP 192.168.53.100 > 192.168
```

- New tun.py successfully pings when the

```
Interface Name: nguye0
IP / ICMP 192.168.53.99 > 192.168.53.100 echo-request 0 / Raw
Sent ICMP echo reply: IP / ICMP 192.168.53.100 > 192.168.53.99 echo-reply 0 / Ra
Traceback (most recent call last):
  File "./tun.py", line 44, in <module>
    os.write(tun, b'This is some arbitrary data')
OSError: [Errno 22] Invalid argument
```

- The error OSError: [Errno 22] Invalid argument occurs
    - This is because the tunnel would be expecting an echo response, not some arbitrary data

Task 3:

```
while True:
        # Get a packet from the tun interface
        packet = os.read(tun, 2048)
        if packet:# Send the packet via the tunnel
                sock.sendto(packet, ("10.9.0.11", 9090))
```

- SERVER_IP and SERVER_PORT are set to the corresponding values, tun_server runs on port 9090, and VPN server has IP 10.9.0.11

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

    packet = os.read(tun, 2048)
KeyboardInterrupt

root@ec5ca555c665:/volumes# tun_client.py
Interface Name: nguye0
^CTraceback (most recent call last):
  File "./tun_client.py", line 32, in <module>
    packet = os.read(tun, 2048)
KeyboardInterrupt

root@ec5ca555c665:/volumes# tun_client.py
Interface Name: nguye0
```

```
root@a0b0e23dfb77:/volumes# ls
tun_client.py  tun_server.py
root@a0b0e23dfb77:/volumes# chmod u+x tun_server.py
root@a0b0e23dfb77:/volumes# tun_server.py
^CTraceback (most recent call last):
  File "./tun_server.py", line 12, in <module>
    data, (ip, port) = sock.recvfrom(2048)
KeyboardInterrupt

root@a0b0e23dfb77:/volumes# tun_server.py
10.9.0.5:39149--> 0.0.0.0:9090
Inside: 192.168.53.99--> 192.168.53.100
```

```
--- 192.168.53.100 ping stati
1 packets transmitted, 0 rece
me 0ms

root@ec5ca555c665:/# ping 192
PING 192.168.53.100 (192.168.
ta.

--- 192.168.53.100 ping stati
1 packets transmitted, 0 rece
me 0ms

root@ec5ca555c665:/#
```

- (left to right terminals) Host U, VPN server, Host U
- This is before a route is added, as you can see the packet cannot be received.

```
root@ec5ca555c665:/# ip route add 192.168.60.0/24 dev nguye0 via
192.168.53.99
```

- Adding ip route (inet of iface nguye0 is 192.168.53.99)

```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

    packet = os.read(tun, 2048)
KeyboardInterrupt

root@ec5ca555c665:/volumes# tun_client.py
Interface Name: nguye0
^CTraceback (most recent call last):
  File "./tun_client.py", line 32, in <module>
    packet = os.read(tun, 2048)
KeyboardInterrupt

root@ec5ca555c665:/volumes# tun_client.py
Interface Name: nguye0
```

```
KeyboardInterrupt

root@a0b0e23dfb77:/volumes# ip route add 192.168.60.
0/24 dev nguye0 via 192.168.60.11
Cannot find device "nguye0"
root@a0b0e23dfb77:/volumes# tun_server.py
10.9.0.5:39149--> 0.0.0.0:9090
Inside: 192.168.53.99--> 192.168.60.3
10.9.0.5:39149--> 0.0.0.0:9090
Inside: 192.168.53.99--> 192.168.60.3
10.9.0.5:39149--> 0.0.0.0:9090
Inside: 192.168.53.99--> 192.168.60.3
```

```
Error: inet address is expected rather
root@ec5ca555c665:/# ip route add 192.
192.168.60.99
Error: Nexthop has invalid gateway.
root@ec5ca555c665:/# ip route add 192.
192.168.53.99
root@ec5ca555c665:/# ping 192.168.60.3
PING 192.168.60.3 (192.168.60.3) 56(84
^C
--- 192.168.60.3 ping statistics ---
3 packets transmitted, 0 received, 100

root@ec5ca555c665:/#
```

- when I ping an IP address in the 192.168.60.0/24 network (in this case, 192.168.60.3), the ICMP packets are received by tun_server.py through the tunnel.

Task 4:

Check if ip forwarding is enabled:

```
root@374b8968eae8:/volumes# cat /proc/sys/net/ipv4/ip_forward
1
```

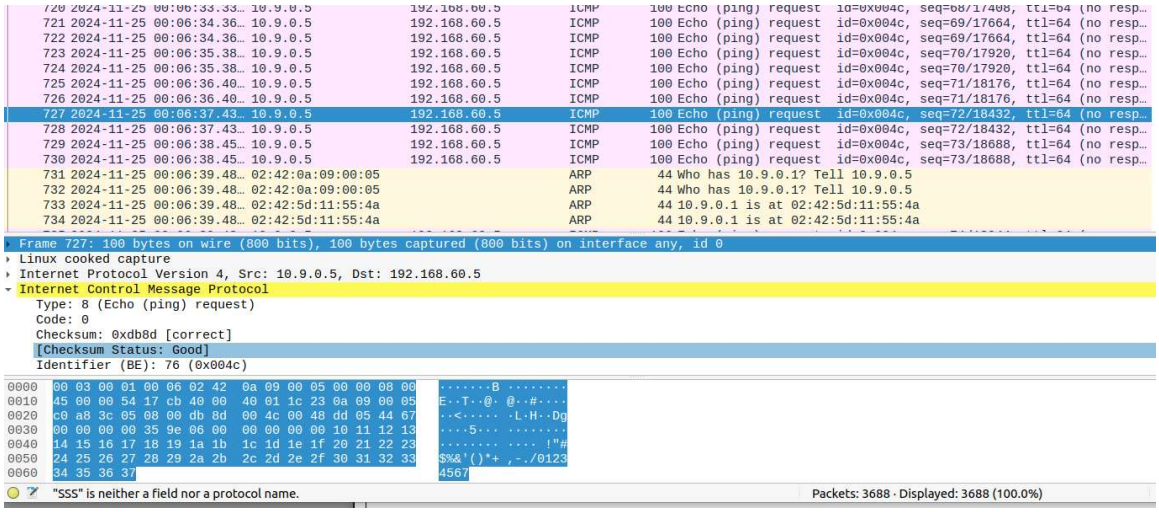
tun_server.py:

```python
ifr = struct.pack('16sH', b'ahmed%d', IFF_TUN | IFF_NO_PI)
ifname_bytes = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")

# Print the TUN interface name
print(f"Interface Name: {ifname}")

# configure the tun interface
os.system(f"ip addr add 192.168.60.1/24 dev {ifname}")
os.system(f"ip link set dev {ifname} up")


# create udp socket
IP_A = "0.0.0.0"  # bind to all available interfaces
PORT = 9090
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((IP_A, PORT))

# receive packets and write them to the tun interface
while True:
    # Receive data from the client
    data, (client_ip, client_port) = sock.recvfrom(2048)
    if data:
        # print received packet info
        print(f"Received packet from {client_ip}:{client_port}")
        pkt = IP(data)
        print(f"Packet: {pkt.src} --> {pkt.dst}")

        # write the received packet to the TUN interface
        os.write(tun, data)
```

Python 3 Tab Width: 8 Ln 23, Col 20

a

```
root@f216db8b062a:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
```



- Wireshark shows (in pink) that the echo request is being sent to host U and shows (in orange) that another address is receiving a request from 10.9.0.5 which must be host V receiving the request since no other requests are occurring at the same time.

Task 5:

Tun_server.py:

```python
#!/usr/bin/env python3

import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'ali%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
print("Interface Name: {}".format(ifname))

# Configure the tun interface
os.system("ip addr add 192.168.53.11/24 dev {}".format(ifname))
```

```python
os.system("ip link set dev {} up".format(ifname))

IP_A = '0.0.0.0'
PORT = 9090

ip, port = '10.9.0.5', 1

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((IP_A, PORT))

while True:
  # this will block until at least one interface is ready
  ready,_,_ = select.select([sock,tun],[],[])

  for fd in ready:
    if fd is sock:
      data, (ip,port) = sock.recvfrom(2048)
      print("From UDP {}:{} --> {}:{}".format(ip, port, IP_A, PORT))
      pkt = IP(data)
      print("From socket(IP) ==>: {} --> {}".format(pkt.src, pkt.dst))
      # ... (code needs to be added by students) ...
      os.write(tun, bytes(pkt))
    if fd is tun:
      packet = os.read(tun,2048)
```

```
    packet = os.read(tun,2048)
    pkt = IP(packet)
    print("From tun ==>: {} --> {}".format(pkt.src, pkt.dst))
    # ... (code needs to be added by students) ...
    sock.sendto(packet, (ip, port))
```

Server:

```
router-10.9.0.11-192.168.60.11:/volumes
$>./tun_server.py
Interface Name: ali0
From UDP 10.9.0.5:50202 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From UDP 10.9.0.5:50202 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From UDP 10.9.0.5:50202 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From UDP 10.9.0.5:50202 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From UDP 10.9.0.5:50202 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From UDP 10.9.0.5:50202 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From UDP 10.9.0.5:50202 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
```

Tun_client.py:

```python
#!/usr/bin/env python3

import fcntl
import struct
import os
import time
from scapy.all import *

# Create UDP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
SERVER_IP, SERVER_PORT = '10.9.0.11', 9090

TUNSETIFF = 0x400454ca
IFF_TUN   = 0x0001
IFF_TAP   = 0x0002
IFF_NO_PI = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes  = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
```

```python
print("Interface Name: {}".format(ifname))

# Configure the tun interface
os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))

# routing
os.system("ip route add 192.168.60.0/24 dev {}".format(ifname))

while True:
  # this will block until at least one interface is ready
  ready,_,_ = select.select([sock,tun],[],[])

  for fd in ready:
    if fd is sock:
      data, (ip,port) = sock.recvfrom(2048)
      print("From UDP {}:{} --> {}".format(ip, port, "10.9.0.5"))
      pkt = IP(data)
      print("From socket(IP) ==>: {} --> {}".format(pkt.src, pkt.dst))
      os.write(tun, bytes(pkt))
    if fd is tun:
      packet = os.read(tun,2048)
      pkt = IP(packet)
      print("From tun ==>: {} --> {}".format(pkt.src, pkt.dst))
```

```python
      print("From tun ==>: {} --> {}".format(pkt.src, pkt.dst))
      sock.sendto(packet, (SERVER_IP, SERVER_PORT))
```

Host:

```
$>./tun_client.py &
[1] 116
U-10.9.0.5:/volumes
$>Interface Name: tun0
$>ping 192.168.60.5 -c 2
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
From tun ==>: 192.168.53.99 --> 192.168.60.5
From UDP 10.9.0.11:9090 --> 10.9.0.5
From socket(IP) ==>: 192.168.60.5 --> 192.168.53.99
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=5.53 ms
From tun ==>: 192.168.53.99 --> 192.168.60.5
From UDP 10.9.0.11:9090 --> 10.9.0.5
From socket(IP) ==>: 192.168.60.5 --> 192.168.53.99
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=9.08 ms

--- 192.168.60.5 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 5.531/7.303/9.076/1.772 ms
U-10.9.0.5:/volumes
```

```
$>tcpdump -i ali0 -n 2>/dev/null &
[2] 127
U-10.9.0.5:/volumes
$>ping 192.168.60.5 -c 2
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
From tun ==>: 192.168.53.99 --> 192.168.60.5
From UDP 10.9.0.11:9090 --> 10.9.0.5
From socket(IP) ==>: 192.168.60.5 --> 192.168.53.99
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=7.57 ms
From tun ==>: 192.168.53.99 --> 192.168.60.5
From UDP 10.9.0.11:9090 --> 10.9.0.5
From socket(IP) ==>: 192.168.60.5 --> 192.168.53.99
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=10.1 ms

--- 192.168.60.5 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 7.569/8.848/10.128/1.279 ms
```

```
U-10.9.0.5:/volumes
$>./tun_client.py &>/dev/null &
[1] 130
U-10.9.0.5:/volumes
$>telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
fec05f0e1dc3 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
```

Task 6:

```
U-10.9.0.5:/volumes
$>telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
fec05f0e1dc3 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Nov 25 03:58:42 UTC 2024 on pts/2
seed@fec05f0e1dc3:~$ date
Mon Nov 25 04:18:04 UTC 2024
seed@fec05f0e1dc3:~$
```

```
^CTraceback (most recent call last):
  File "./tun_server.py", line 37, in <module>
    ready,_,_ = select.select([sock,tun],[],[])
KeyboardInterrupt

router-10.9.0.11-192.168.60.11:/volumes
$>./tun_server.py
Interface Name: ali0
From UDP 10.9.0.5:56506 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From UDP 10.9.0.5:56506 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From UDP 10.9.0.5:56506 --> 0.0.0.0:9090
From socket(IP) ==>: 192.168.53.99 --> 192.168.60.5
From UDP 10.9.0.5:56506 --> 0.0.0.0:9090
```

```
To restore this content, you can run the 'unminimize' command.
Last login: Mon Nov 25 03:58:42 UTC 2024 on pts/2
seed@fec05f0e1dc3:~$ date
Mon Nov 25 04:18:04 UTC 2024
seed@fec05f0e1dc3:~$ date
```

- When we were connected to Host V from Host U using a telnet session we were able to run the dat command and the output was returned
- When we disrupted the VPN tunnel and the connection was broken, the date command we typed didn't show up and was not sent to Host V but the telnet session was not terminated immediately
- When we restarted the scripts and restored the VPN tunnel the connections resumed without resetting and the date command was sent and the response was returned

Task 7:

```
[11/24/24]seed@VM:~/.../Labsetup$ dockps
3381348b2afa  client-10.9.0.5
fec05f0e1dc3  host-192.168.60.5
a8921c371ff9  server-router
585b344be01e  host-192.168.60.6
[11/24/24]seed@VM:~/.../Labsetup$ docksh host-192.168.60.5
root@fec05f0e1dc3:/# ip route
default via 192.168.60.11 dev eth0
192.168.60.0/24 dev eth0 proto kernel scope link src 192.168.60.5
root@fec05f0e1dc3:/# ip route del default
root@fec05f0e1dc3:/# ip route
192.168.60.0/24 dev eth0 proto kernel scope link src 192.168.60.5
root@fec05f0e1dc3:/# ip route add 192.168.53.0/24 via 192.168.60.1
root@fec05f0e1dc3:/# ip route
192.168.53.0/24 via 192.168.60.11 dev eth0
192.168.60.0/24 dev eth0 proto kernel scope link src 192.168.60.5
root@fec05f0e1dc3:/#
```

- We deleted the default route so Host V cannot send packets outside the local network (192.168.60.0/24) by default
- Added a specific route directing traffic for the VPN client's network (192.168.53.0/24) to go through the VPN server (192.168.60.11)
- This ensures packets to/from the VPN client flow correctly through the VPN

server and tunnel

Task 8:

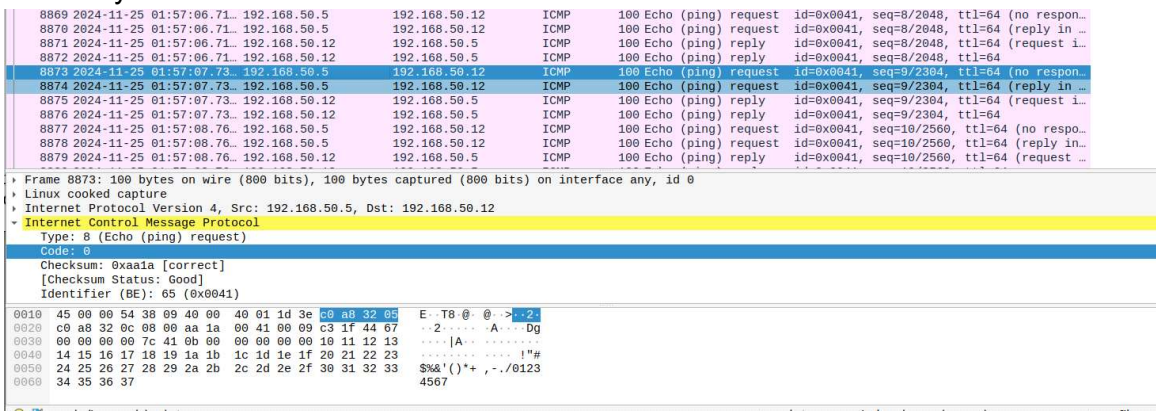
After running tun_client.py from host U we can see the routes from host U which shows a connection to the tunnel:

```
root@b42bf6c58666:/# ip route show
default via 10.9.0.1 dev eth0
10.9.0.0/24 dev eth0 proto kernel scope link src 10.9.0.12
192.168.50.0/24 dev eth1 proto kernel scope link src 192.168.50.12
192.168.53.0/24 dev ahmed0 proto kernel scope link src 192.168.53.1
root@b42bf6c58666:/#
```

Pinging the vpn from host V works which indicates that a connection is established:

```
root@8c6e90ff3aa9:/# ping 192.168.60.11
PING 192.168.60.11 (192.168.60.11) 56(84) bytes of data.
64 bytes from 192.168.60.11: icmp_seq=1 ttl=64 time=0.062 ms
64 bytes from 192.168.60.11: icmp_seq=2 ttl=64 time=0.052 ms
64 bytes from 192.168.60.11: icmp_seq=3 ttl=64 time=0.052 ms
64 bytes from 192.168.60.11: icmp_seq=4 ttl=64 time=0.053 ms
```

lets verify this in wireshark:

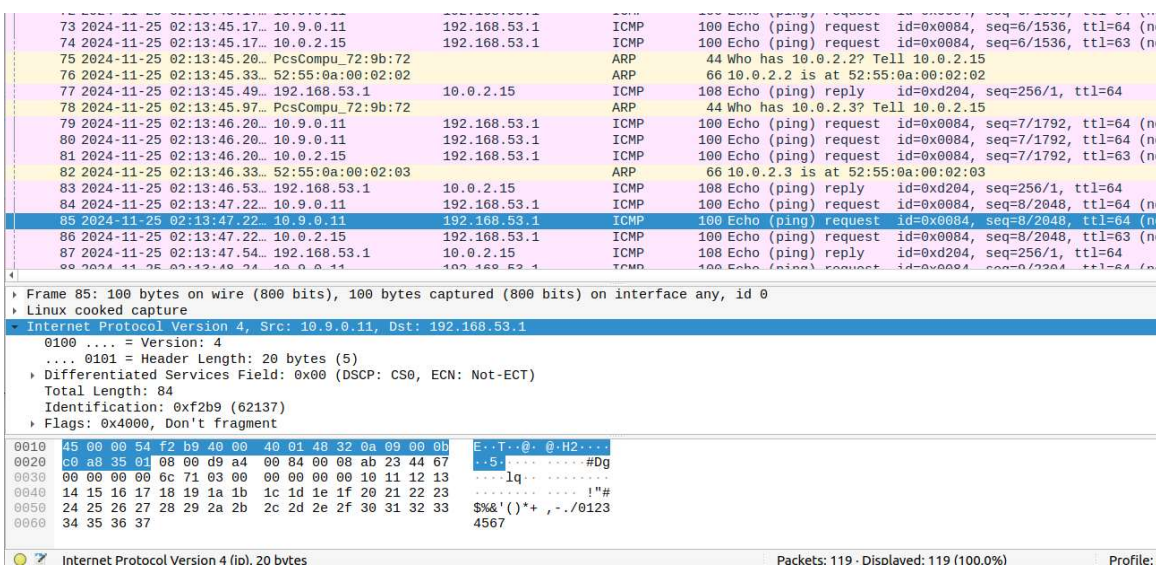


Task 9:

tap_interface.py:

```python
#!/usr/bin/env python3
import os
import fcntl
import struct
import sys
from scapy.layers.l2 import Ether, ARP

tap = os.open("/dev/net/tun", os.O_RDWR)

ifr = struct.pack('16sH', b'tap%d', 0x4002)  # 0x4002 is IFF_TAP | IFF_NO_PI
ifname_bytes = fcntl.ioctl(tap, 0x400454ca, ifr)  # TUNSETIFF
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")

print(f"TAP interface created: {ifname}")

while True:
    packet = os.read(tap, 2048)
    if packet:
        ether = Ether(packet)
        print(ether.summary())

```

```
root@374b8968eae8:/volumes# ping 192.168.53.1
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.
^C
--- 192.168.53.1 ping statistics ---
16 packets transmitted, 0 received, 100% packet loss, time 15352ms
```



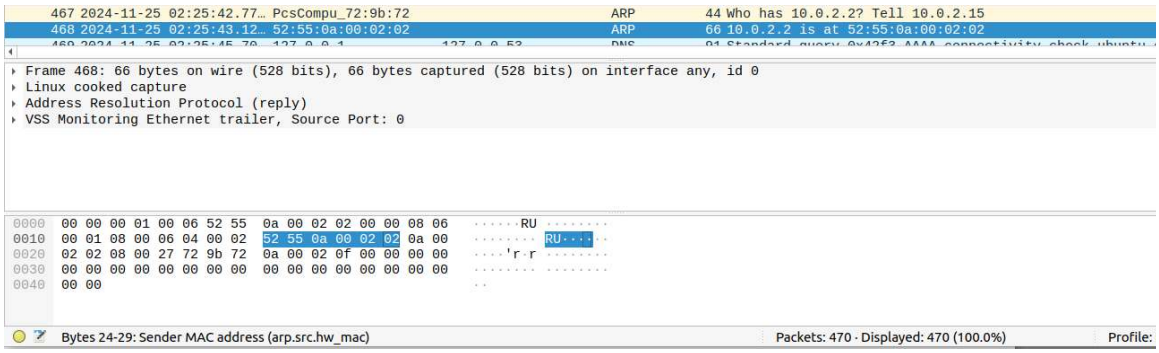
It seems that when pinging host U from the vpn, the echo request is sent and replied although it is only detected in wireshark. Now lets test with ARP:

Updated tap_interface.py:

```python
#!/usr/bin/env python3
import os
import fcntl
import struct
import sys
from scapy.layers.l2 import Ether, ARP

tap = os.open("/dev/net/tun", os.O_RDWR)

ifr = struct.pack('16sH', b'tap%d', 0x4002)  # 0x4002 is IFF_TAP | IFF_NO_PI
ifname_bytes = fcntl.ioctl(tap, 0x400454ca, ifr)  # TUNSETIFF
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")

print(f"TAP interface created: {ifname}")

while True:
    packet = os.read(tap, 2048)
    if packet:
        ether = Ether(packet)
        print(ether.summary())
        if ARP in ether and ether[ARP].op == 1:
            arp = ether[ARP]
            newether = Ether(dst=ether.src, src="aa:bb:cc:dd:ee:ff")
            newarp = ARP(psrc=arp.pdst, hwsrc="aa:bb:cc:dd:ee:ff",
                         pdst=arp.psrc, hwdst=ether.src, op=2)
            newpkt = newether / newarp
            print("***** Fake response: {}".format(newpkt.summary()))
            os.write(tap, bytes(newpkt))

```

```
root@374b8968eae8:/# arping -I tap0 192.168.53.33
ARPING 192.168.53.33
Timeout
Timeout
Timeout
Timeout
Timeout
^C
--- 192.168.53.33 statistics ---
6 packets transmitted, 0 packets received, 100% unanswered (0 extra)
```

It seems that the arp request times out in thhe terminal, however in wireshark, we get an ARP response as seen below:


467 2024-11-25 02:25:42.77… PcsCompu_72:9b:72 ARP 44 Who has 10.0.2.2? Tell 10.0.2.15
468 2024-11-25 02:25:43.12… 52:55:0a:00:02:02 ARP 66 10.0.2.2 is at 52:55:0a:00:02:02
Frame 468: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface any, id 0
Linux cooked capture
Address Resolution Protocol (reply)
VSS Monitoring Ethernet trailer, Source Port: 0
0000 00 00 00 01 00 06 52 55 0a 00 02 02 00 00 08 06 ·····RU ·····
0010 00 01 08 00 06 04 00 02 52 55 0a 00 02 02 0a 00 ····· RU·····
0020 02 02 08 00 27 72 9b 72 0a 00 02 0f 00 00 00 00 ····'r·r ·····
0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ····· ·····
0040 00 00 ··
Bytes 24-29: Sender MAC address (arp.src.hw_mac)
Packets: 470 · Displayed: 470 (100.0%)
Profile: